

연세대학교  
글로벌인재대학



베트남 증권시장에서  
주가 예측과 지능형 포트폴리오 구축에  
선형대수학의 응용

수업: 인공지능기초수학 (GAI3002.01-00)

성명 : Nguyen Ha Chi

학번 : 2024106126

전공 : 국제통상 & 응용정보공학

# 목차

|          |                         |           |
|----------|-------------------------|-----------|
| <b>1</b> | <b>개발 배경</b>            | <b>2</b>  |
| 1.1      | 포트폴리오 최적화 문제            | 2         |
| 1.2      | 프로젝트 소개                 | 2         |
| <b>2</b> | <b>개요</b>               | <b>2</b>  |
| 2.1      | 시스템 아키텍처                | 2         |
| 2.2      | 입력 및 출력                 | 3         |
| 2.3      | 파이프라인 구성                | 3         |
| <b>3</b> | <b>기능 및 코드</b>          | <b>4</b>  |
| 3.1      | 데이터 수집 및 처리 (NB01)      | 4         |
| 3.2      | 수익률 및 리스크 측정 (NB02)     | 6         |
| 3.3      | 주성분 분석 — PCA (NB03)     | 8         |
| 3.3.1    | 수학적 기반                  | 8         |
| 3.3.2    | 핵심 코드                   | 8         |
| 3.3.3    | 세 가지 잠재 요인              | 9         |
| 3.4      | 기대 수익률 예측 (NB04, NB05)  | 10        |
| 3.4.1    | 이동평균법 (NB04)            | 10        |
| 3.4.2    | 핵심 코드                   | 10        |
| 3.5      | 다중선형회귀 예측 — MLR (NB05)  | 12        |
| 3.5.1    | 수학적 기반                  | 12        |
| 3.5.2    | 핵심 코드                   | 12        |
| 3.5.3    | Moving Average 와의 비교    | 13        |
| 3.6      | Sharpe ratio 최적화 (NB06) | 14        |
| 3.6.1    | 수학적 기반                  | 14        |
| 3.6.2    | 핵심 코드                   | 14        |
| 3.6.3    | 결과                      | 15        |
| <b>4</b> | <b>문제점과 해결 과정</b>       | <b>16</b> |
| 4.1      | 아웃오브샘플 백테스트             | 16        |
| 4.2      | 과적합 및 추정 오차 분석          | 17        |
| 4.3      | 추정 오차의 영향               | 18        |
| <b>5</b> | <b>한계 및 개선 방향</b>       | <b>18</b> |
| 5.1      | 현재 모델의 한계               | 18        |
| 5.2      | 개선 방향                   | 18        |
| 5.3      | 결론                      | 19        |
|          | <b>참고문헌</b>             | <b>20</b> |

## 1 개발 배경

### 1.1 포트폴리오 최적화 문제

포트폴리오 최적화란 일정 수준의 위험 하에서 기대 수익을 극대화하는 것을 목표로 금융 자산에 자본을 배분하는 과정이다. 포트폴리오 구축 과정은 두 가지 기본 단계로 구성된다. 첫째, 자산의 미래 행동에 대한 기대치를 추정하기 위해 과거 데이터를 분석한다. 둘째, 해당 추정치를 바탕으로 배분 비중을 최적화한다.

포트폴리오의 위험은 개별 자산의 위험을 단순 합산한 것과 같지 않으며, 자산 간의 상관관계에 따라 결정된다. 서로 반대 방향으로 움직이는 경향이 있는 두 자산을 결합하면 전체 위험이 상쇄되어, 투자자는 더 낮은 위험으로 동일한 수준의 수익을 달성할 수 있다. 예를 들어, 주식 A가 오를 때 주식 B가 내리는 경향이 있다면, 두 종목을 동시에 보유하는 것이 A 또는 B 만 단독으로 보유하는 것보다 안정적이다. 이처럼 두 자산은 서로를 보완하여 시장 변동에 대한 포트폴리오의 민감도를 낮춘다. 따라서 투자자는 여러 종목을 동시에 매수하여 포트폴리오를 구성하며, 그 목표는 최소한의 위험으로 최대한의 수익을 달성하는 것이다.

수익과 위험 간의 상충 관계를 정량화하기 위해 널리 사용되는 지표가 Sharpe ratio이며, 다음과 같이 정의된다.

$$\text{Sharpe} = \frac{\mu^\top w - r_f}{\sqrt{w^\top \Sigma w}} \quad (1)$$

여기서  $w \in \mathbb{R}^n$  은 자본 배분 비중 벡터,  $\mu \in \mathbb{R}^n$  은 자산의 기대 수익률 벡터,  $\Sigma \in \mathbb{R}^{n \times n}$  은 자산 간 위험 및 상관관계를 나타내는 공분산 행렬,  $r_f$  는 무위험 이자율이다. Sharpe ratio 가 높을수록 포트폴리오가 수용하는 위험 한 단위당 더 많은 수익을 창출한다는 것을 의미하며, 이는 투자 전략 간 성과를 비교하는 표준 지표로 활용된다.

따라서 투자자의 목표는 전체 자금의 100% 를 각 종목에 어떻게 배분할지를 결정하여 Sharpe ratio 를 최대화하는 것이다.

### 1.2 프로젝트 소개

본 프로젝트는 베트남 증권시장, 구체적으로는 호치민 증권거래소 (HOSE, Ho Chi Minh Stock Exchange) 에 상장된 8 개의 블루칩 종목을 대상으로 포트폴리오 최적화 프로세스를 적용한다. 분석 대상 종목은 VNM, VIC, VHM, FPT, HPG, MWG, VCB, MBB 이며, 이들은 모두 베트남을 대표하는 대형 우량기업으로서 안정적인 거래 이력과 높은 유동성을 갖추고 있다.

## 2 개요

### 2.1 시스템 아키텍처

본 시스템은 원시 데이터에서 최적 포트폴리오까지, 네 가지 주요 단계로 구성된 선형 파이프라인으로 조직된다.

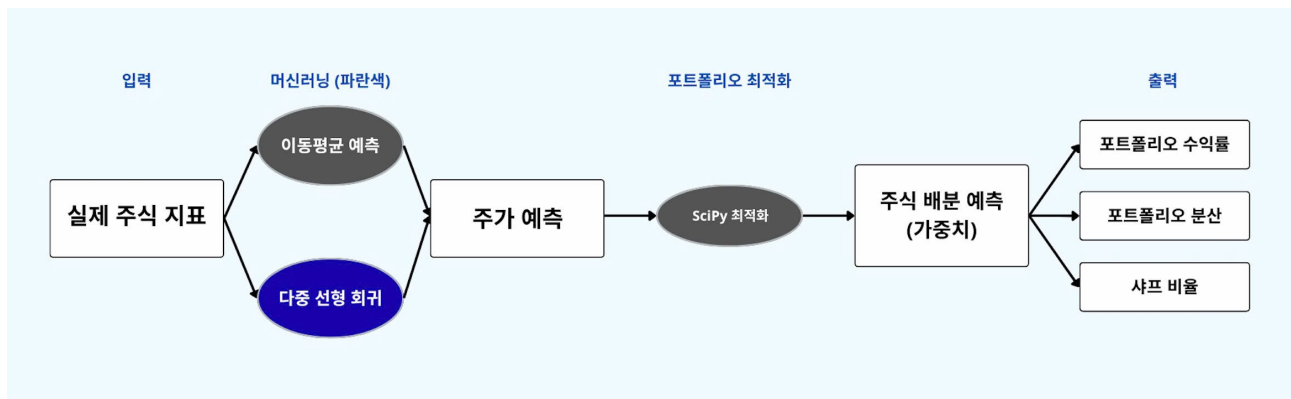


Figure 1: 시스템 아키텍처 다이어그램

본 시스템은 8 개 종목의 주가 이력을 입력으로 받아, 각 종목의 기대 수익률, 위험도, 종목 간 상관관계 등 필요한 파라미터를 산출한다. 이를 바탕으로 최적화 문제를 풀어 자본을 가장 효율적으로 배분하는 방법을 도출한다.

## 2.2 입력 및 출력

**입력 (Input):** 2018 년부터 2023 년까지 HOSE(호치민 증권거래소) 에 상장된 8 개 블루칩 종목의 일별 종가 데이터로, vnstock 라이브러리를 통해 수집하였다. 분석 대상 종목은 다음과 같다.

| 종목코드 | 기업명             | 업종  |
|------|-----------------|-----|
| VNM  | Vinamilk        | 소비재 |
| VIC  | Vingroup        | 부동산 |
| VHM  | Vinhomes        | 부동산 |
| FPT  | FPT Corporation | 기술  |
| HPG  | Hoa Phat Group  | 철강  |
| MWG  | Mobile World    | 유통  |
| VCB  | Vietcombank     | 은행  |
| MBB  | MB Bank         | 은행  |

Table 1: 분석 대상 8 개 종목

원시 가격 데이터는 모델에 투입되기 전에 일별 수익률 (daily returns) 로 변환된다.

**출력 (Output):** 각 종목에 대한 자본 배분 비율을 나타내는 비중 벡터  $w = [w_1, w_2, \dots, w_8]^T$

## 2.3 파이프라인 구성

본 프로젝트는 총 6 개의 노트북으로 구성된 순차적 파이프라인으로 구현된다.

| 노트북  | 내용                                   |
|------|--------------------------------------|
| NB01 | 데이터 수집 및 처리: 주가 다운로드 및 수익률 계산        |
| NB02 | 기대 수익률 벡터 $\mu$ 및 공분산 행렬 $\Sigma$ 산출 |
| NB03 | 주성분 분석 (PCA)                         |
| NB04 | 이동평균법 (MA5) 을 이용한 $\mu$ 추정           |
| NB05 | 다중선형회귀 (MLR) 를 이용한 $\mu$ 추정          |
| NB06 | Sharpe ratio 최적화 문제 풀이 및 결과 평가       |

Table 2: 프로젝트 파이프라인 구성

### 3 기능 및 코드

#### 3.1 데이터 수집 및 처리 (NB01)

첫 번째 단계는 베트남의 8 개 대형 상장 주식의 역사적 종가 데이터를 수집하고, 이를 시간 축을 기준으로 하나의 공통 데이터셋으로 통합한 후, 결측값을 처리하여 데이터의 일관성을 확보하는 것이다.

Listing 1: 종목 데이터 수집 코드 (NB01)

```
# List of 8 HOSE blue-chip stocks
tickers = ['VNM', 'VIC', 'VHM', 'FPT', 'HPG', 'MWG', 'VCB', 'MBB']

# Time range: 3 recent years
start_date = '2022-01-01'
end_date   = '2025-05-01'

# Dictionary to store closing prices of each stock
all_prices = {}

for ticker in tickers:
    print(f"Fetching data: {ticker}...")
    try:
        q = Quote(symbol=ticker, source='VCI')
        df = q.history(start=start_date, end=end_date, interval='1D')
        df['time'] = pd.to_datetime(df['time'])
        all_prices[ticker] = df.set_index('time')['close']
        print(f"    -> Loaded {len(df)} trading days")
    except Exception as e:
        print(f"    -> Error: {e}")

# Combine all into a single DataFrame
prices_df = pd.DataFrame(all_prices)
print(f"\nSummary: {prices_df.shape[0]} days x {prices_df.shape[1]}")
```

```
stocks")
print("\nFirst 5 rows:")
print(prices_df.head())
print("\nLast 5 rows:")
print(prices_df.tail())
```

데이터 수집 기간은 최근 3 년으로 설정하였다. 기간이 길수록 통계 추정에 유리하지만, 지나치게 오래된 데이터는 현재 시장 구조를 반영하지 못할 수 있다. 약 3 년은 통계적 유의성을 확보하면서도 현재 시장과의 관련성을 유지하는 합리적인 균형점이다.

이후 데이터는 `dropna()` 를 통해 결측값을 제거하고 정제된 후 `prices.csv` 파일로 저장되어 이후 노트북에서 활용된다. 처리 후 데이터셋의 주요 특성은 다음과 같다.

| 항목      | 값                                 |
|---------|-----------------------------------|
| 종목 수    | 8                                 |
| 거래일 수   | 871 일                             |
| 기간      | 2021 년 11 월 1 일 – 2025 년 4 월 29 일 |
| 결측값 수   | 0                                 |
| 제거된 거래일 | 0                                 |

Table 3: 처리 후 데이터셋 요약

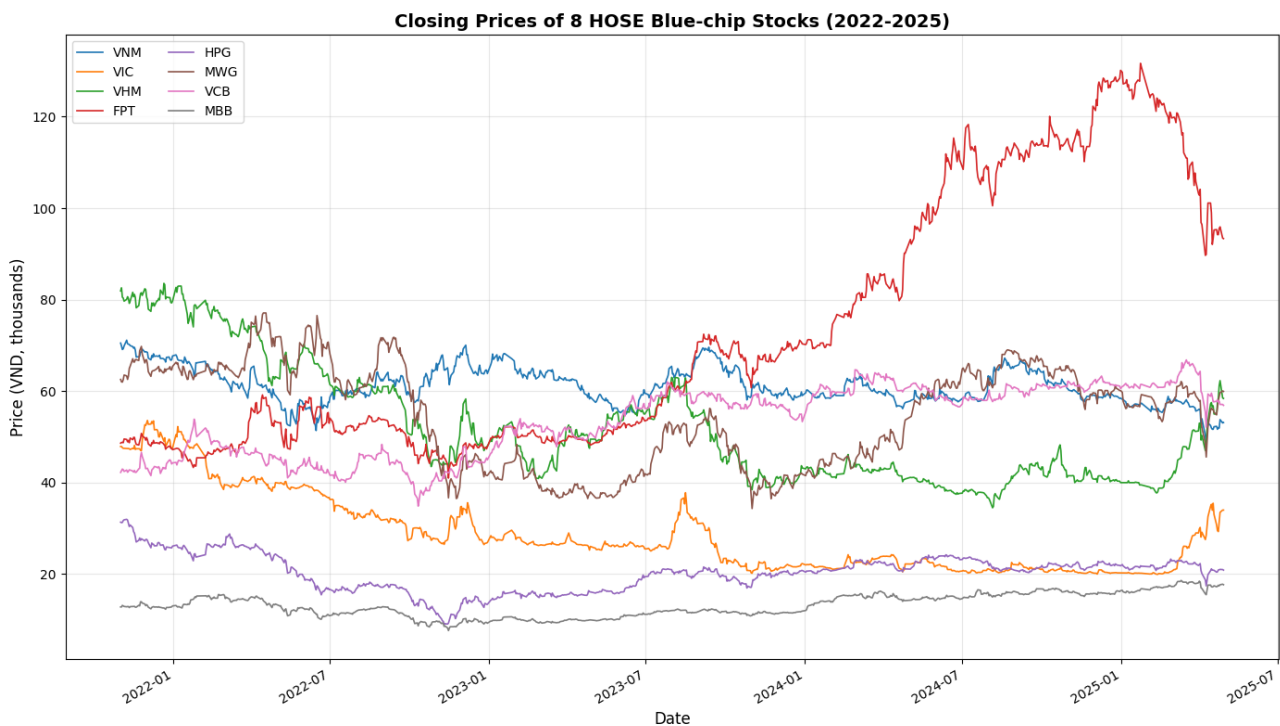


Figure 2: 8 개 HOSE 블루칩 종목의 종가 추이 (2022–2025)

그림 2에서 알 수 있듯이, 종목별 주가 수준은 상당한 차이를 보인다. FPT의 종가가 가장 높아 2024년 말에서 2025년 초 사이에 120,000–130,000 동에 달했으며, MBB는 10,000–20,000 동 수

준으로 가장 낮았다. 나머지 종목들은 40,000–80,000 동 사이에 분포하였다. 또한 각 종목은 서로 다른 가격 추세를 보이며, 상승·하락·횡보가 혼재하는 양상을 나타낸다.

### 3.2 수익률 및 리스크 측정 (NB02)

앞서 살펴본 바와 같이 종목별 주가 수준은 크게 차이가 나므로 (FPT 는 100,000 동 이상, MBB 는 10,000 동 수준) 직접 비교가 불가능하다. 따라서 매일의 등락률, 즉 일별 수익률로 변환하여 분석한다. 이 경우 100 에서 102 로 오른 종목 (+2%) 과 10 에서 10.2 로 오른 종목 (+2%) 은 동일하게 취급된다.

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}} \quad (2)$$

첫째 날은 비교할 전일 가격이 없으므로 제외되며, 최종적으로 870 일치 데이터가 사용된다.

다음으로 각 종목의 기대 수익률과 리스크를 산출한다. 기대 수익률은 일별 수익률의 평균이며, 리스크는 가격 변동의 크기를 나타내는 표준편차로 측정한다. 일별 수치는 매우 작아 직관적으로 이해하기 어려우므로, 연율화하여 은행 금리와 같이 익숙한 기준으로 변환한다. 이 두 지표를 하나의 지수로 통합한 것이 Sharpe ratio 로, 수익을 리스크로 나눈 값이다. Sharpe ratio 가 높을수록 적은 위험으로 많은 수익을 창출하는 종목임을 의미한다.

Listing 2: 연율화 수익률 및 리스크 산출 코드 (NB02)

```
# Annualization factor: HOSE has ~250 trading days/year
# (verified empirically from our 2022-2024 full-year data: 249, 249, 250)
TRADING_DAYS = 250

ann = pd.DataFrame({
    'ann_return_%': returns.mean() * TRADING_DAYS * 100,
    'ann_volatility_%': returns.std() * np.sqrt(TRADING_DAYS) * 100,
})

ann['sharpe_rf0'] = ann['ann_return_%'] / ann['ann_volatility_%']
ann = ann.round(3).sort_values('sharpe_rf0', ascending=False)
```

| 종목  | 연 수익률 (%) | 연 변동성 (%) | Sharpe |
|-----|-----------|-----------|--------|
| FPT | 22.389    | 27.003    | 0.829  |
| VCB | 11.407    | 23.870    | 0.478  |
| MBB | 13.745    | 29.986    | 0.458  |
| MWG | 5.235     | 35.915    | 0.146  |
| VHM | -4.748    | 31.569    | -0.150 |
| VIC | -4.866    | 31.664    | -0.154 |
| HPG | -5.756    | 34.483    | -0.167 |
| VNM | -5.682    | 22.169    | -0.256 |

Table 4: 종목별 연율화 수익률, 변동성 및 Sharpe ratio

결과를 보면 8 개 종목 중 4 개만이 양의 수익률을 기록하였다. FPT 가 연 약 22% 의 수익률과 최고 Sharpe ratio(0.83) 로 압도적인 우위를 보였으며, 은행주인 VCB 와 MBB 가 그 뒤를 이었다. 나머지 4 개 종목은 손실을 기록하며 해당 기간 업종의 어려움을 반영하였다. VNM 은 변동성이 가장 낮았음에도 Sharpe ratio 가 가장 낮았는데, 이는 리스크가 낮더라도 손실이 발생하면 의미가 없음을 보여준다.

다음으로 종목 간 동조화 여부를 파악하기 위해 공분산 행렬  $\Sigma$  를 산출한다. 이는 각 종목 쌍이 같은 방향으로 움직이는지, 반대 방향으로 움직이는지, 그리고 그 강도는 어느 정도인지를 나타낸다.

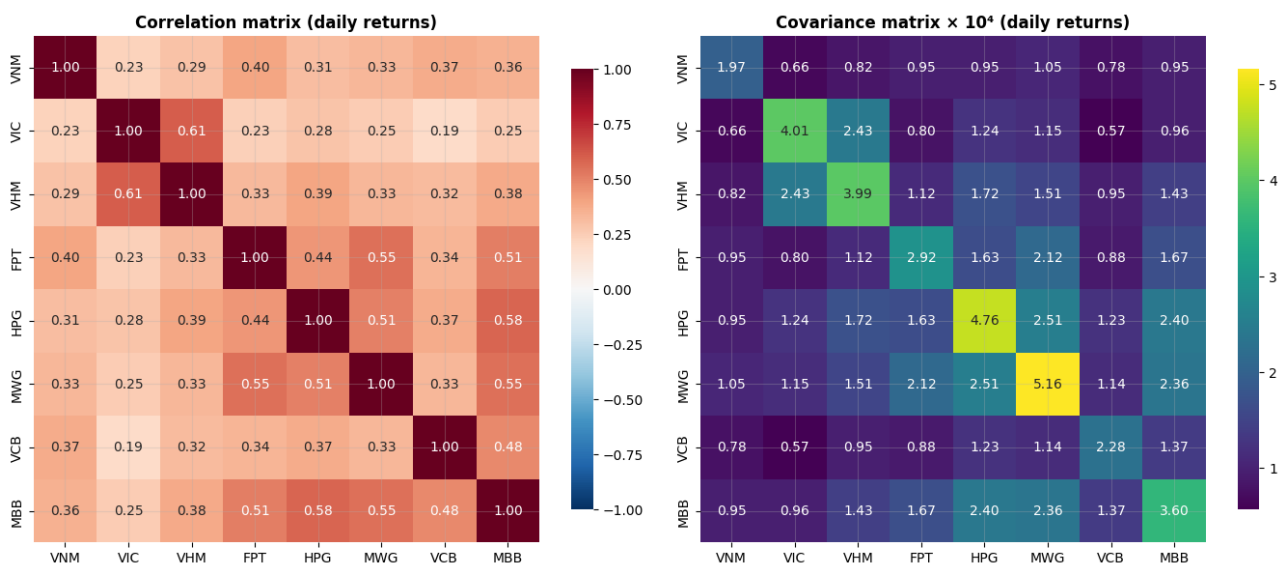


Figure 3: 8 개 종목의 상관관계 행렬 (좌) 및 공분산 행렬 (우)

상관관계 행렬을 살펴보면, VIC 와 VHM 이 가장 강한 동조화 (0.61) 를 보이는데, 이는 Vinhomes(VHM) 가 Vingroup(VIC) 의 자회사이기 때문이다. 또한 모든 종목 쌍이 양의 상관관계를 보이며 반대 방향으로 움직이는 쌍은 존재하지 않는다. 이는 베트남과 같은 신흥 시장의 전형적인 특징으로, 금리·환율·외국인 자금 흐름 등 공통 요인의 영향을 거의 모든 종목이 동시에 받기 때문이다. 따라서 분산투자의 효과는 종목 간 상쇄 효과가 아닌, 변동성이 낮은 종목에 더 많은



비중을 배분하는 방식으로만 실현될 수 있다.

### 3.3 주성분 분석 — PCA (NB03)

앞서 8 개 종목이 서로 연관된 움직임을 보인다는 사실을 확인하였다. 이는 8 개의 개별 움직임 뒤에 이들을 동시에 지배하는 몇 가지 공통 “힘” 이 숨어 있을 가능성을 시사한다. 주성분 분석 (PCA) 은 바로 이 숨겨진 힘을 찾아내는 도구다. PCA 는 소수의 주요 “변동 방향” 을 찾아내며, 각 방향은 하나의 잠재 요인을 나타낸다. 단 몇 개의 요인만으로도 8 개 종목 전체 변동의 대부분을 설명할 수 있다.

#### 3.3.1 수학적 기반

PCA 의 핵심은 공분산 행렬  $\Sigma$  에 대한 **고유분해 (eigendecomposition)** 이다.  $\Sigma$  는 대칭 행렬이자 양반정치 행렬이므로, 선형대수학의 스펙트럼 정리 (spectral theorem) 에 의해 항상 다음과 같이 분해할 수 있다.

$$\Sigma = V \Lambda V^T \quad (3)$$

여기서  $V$  는 직교 행렬로, 각 열이 하나의 **고유벡터 (eigenvector)** 이며,  $\Lambda$  는 대각 행렬로 대각 원소가 해당 **고유값 (eigenvalue)**  $\lambda_i$  이다. 각 고유벡터는 8 차원 주식 공간에서 하나의 방향, 즉 하나의 잠재 요인에 해당한다. 고유값  $\lambda_i$  는 그 방향으로의 분산 크기를 나타내므로, 고유값이 클수록 해당 요인이 시장 전체에서 더 중요하다는 의미다.  $V^T V = I$  라는 직교성 조건은 각 잠재 요인이 서로 완전히 독립적임을 보장하므로, 어떤 요인도 다른 요인의 정보를 중복하여 담지 않는다.

#### 3.3.2 핵심 코드

PCA 의 계산 핵심은 단 몇 줄로 구현된다.  $\Sigma$  가 대칭 행렬이므로 `np.linalg.eigh` 함수를 사용하며, 이 함수는 고유값을 오름차순으로 반환하므로 가장 중요한 요인이 맨 앞에 오도록 순서를 뒤집는다.

Listing 3: 공분산 행렬의 고유분해 (NB03)

```
# Eigendecomposition of Sigma
# Note: eigh assumes Sigma is symmetric (which we verified in Section 2)
eigenvalues_raw, eigenvectors_raw = np.linalg.eigh(Sigma)

# eigh returns eigenvalues in ASCENDING order
# For PCA we want DESCENDING (largest variance first)
sort_idx = np.argsort(eigenvalues_raw)[::-1]
eigenvalues = eigenvalues_raw[sort_idx]
eigenvectors = eigenvectors_raw[:, sort_idx]
```

이 코드를 실행하면 8 개의 고유값이 큰 순서대로 출력된다. 주목할 점은 고유값들 사이의 격차다. 첫 번째 고유값  $\lambda_1$  은 두 번째의 약 3 배, 마지막 고유값들의 10 배 이상에 달한다. 고유값은 각 잠재 요인이 설명하는 분산의 크기이므로, 이 큰 격차는 단 하나의 지배적인 요인이 8 개 종목

전체 움직임을 압도적으로 설명하고 있음을 의미한다. 시각화나 각 요인의 의미 해석 이전에, 고유값 수열만 보더라도 베트남 주식 시장이 하나의 강력한 공통 요인에 의해 움직이고 있음을 알 수 있다.

### 3.3.3 세 가지 잠재 요인

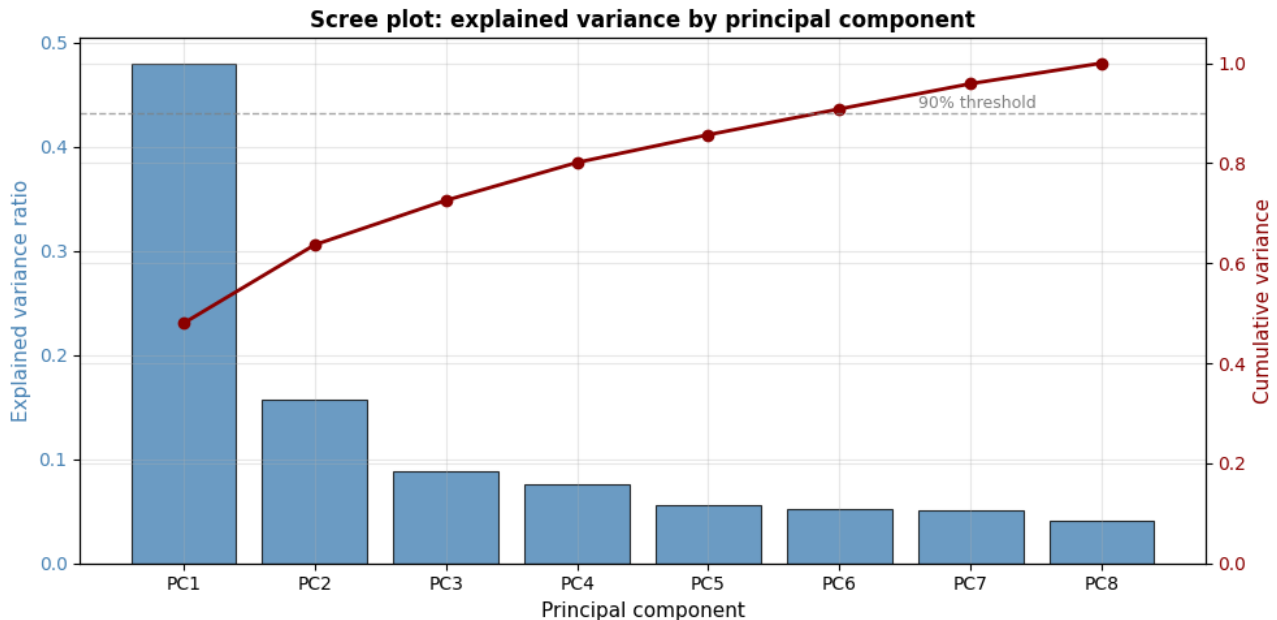


Figure 4: 스크리 플롯: 주성분별 설명 분산 비율

그림 4에서 알 수 있듯이, 상위 3 개의 주성분만으로 8 개 종목 전체 변동의 약 73% 를 설명할 수 있다.

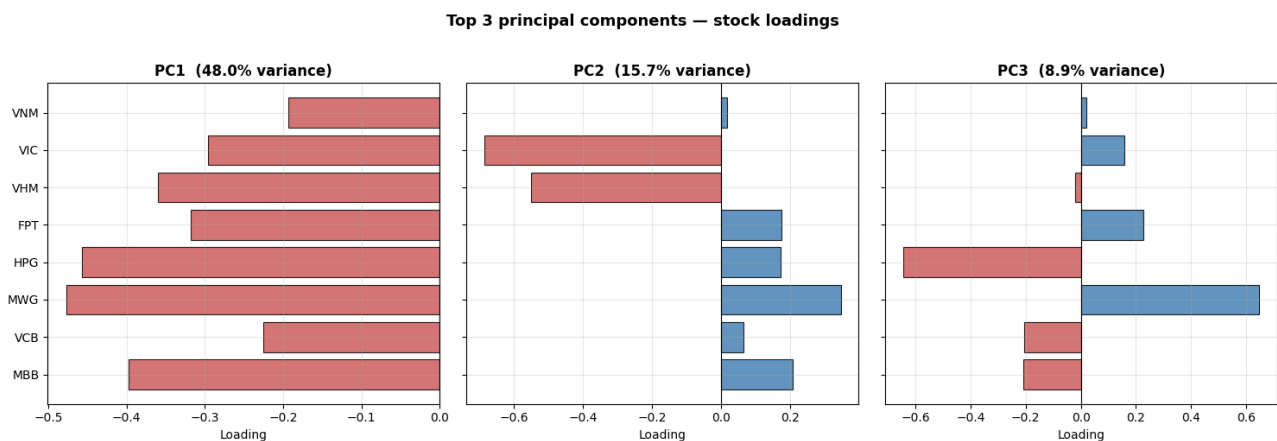


Figure 5: 상위 3 개 주성분에 대한 종목별 기여도

그림 5의 각 막대는 해당 종목이 각 잠재 요인에 얼마나 기여하는지를 나타낸다. 세 요인은 다음과 같이 해석된다.

**PC1 (48%): 시장 공통 요인.** 8 개 종목 모두 같은 방향으로 움직인다. 베트남 시장 전체가 오르거나 내릴 때 블루칩 종목들이 함께 움직이는 현상을 포착한 것으로, 분산투자자로도 제거할 수 없는 시장 전체 리스크에 해당한다.

**PC2 (16%): Vingroup 요인.** VIC 와 VHM 만 반대 방향으로 분리된다. Vinhomes(VHM) 가 Vingroup(VIC) 의 자회사이므로, 두 종목이 그룹 특유의 요인에 의해 함께 움직이는 현상을 반영한다. 이는 앞서 두 종목의 상관계수가 0.61 로 가장 높았던 결과와 일치한다.

**PC3 (9%): 철강-유통 대립 요인.** HPG(철강) 와 MWG(유통) 가 서로 반대 방향으로 움직인다. 두 업종이 서로 다른 경제적 요인 (원자재 가격, 내수 소비) 에 의존하기 때문이다.

8 개 HOSE 블루칩 종목의 복잡한 움직임은 세 가지 직관적인 잠재 요인, 즉 시장 공통 요인, Vingroup 요인, 철강-유통 대립 요인으로 요약된다. 특히 시장 공통 요인이 전체의 절반 가량을 차지하며, 분산투자로 제거할 수 없다는 사실은 베트남 주식 포트폴리오에서 리스크 감소 여지가 제한적임을 시사한다.

### 3.4 기대 수익률 예측 (NB04, NB05)

최적화 문제를 풀기 위해서는 각 종목이 앞으로 얼마나 수익을 낼지, 즉 기대 수익률  $\mu$  를 추정해야 한다. 본 프로젝트는 두 가지 방법을 사용한다. 하나는 단순 비교 기준선으로 활용하는 이동평균법 (Moving Average) 이고, 다른 하나는 보다 정교한 다중선형회귀 (MLR) 이다. 본 절에서는 첫 번째 방법을 설명한다.

#### 3.4.1 이동평균법 (NB04)

이동평균법은 가장 최근  $w$  일간의 가격 평균을 다음 날 가격 예측값으로 사용하는 방법이다. 예를 들어 MA5 는 가장 최근 5 일간의 평균 가격을 내일의 예측값으로 사용한다.

$$\hat{P}_{t+1} = \frac{1}{w} \sum_{i=0}^{w-1} P_{t-i} \quad (4)$$

#### 3.4.2 핵심 코드

이동평균 예측 함수는 먼저 지정된 일수만큼의 가격 이동 평균을 계산한 뒤, 결과를 하루 뒤로 이동 (shift) 시킨다. 이 이동 처리는 오늘의 예측값이 오직 어제까지의 데이터만을 사용하도록 보장하여, look-ahead bias(미래 데이터를 미리 사용하는 오류) 를 방지한다.

Listing 4: 이동평균 예측 함수 (NB04)

```
def predict_with_ma(prices_df, window):
    # MA computed at time t uses prices up to and including t
    # We shift by 1 so that the value at row t is the prediction
    # made AT time t-1 using information available up to t-1.
    ma = prices_df.rolling(window=window).mean()
    predictions = ma.shift(1)
    return predictions
```

MA5, MA20, MA50 을 실제 가격과 함께 시각화하면 각 방법의 특성을 명확히 확인할 수 있다.

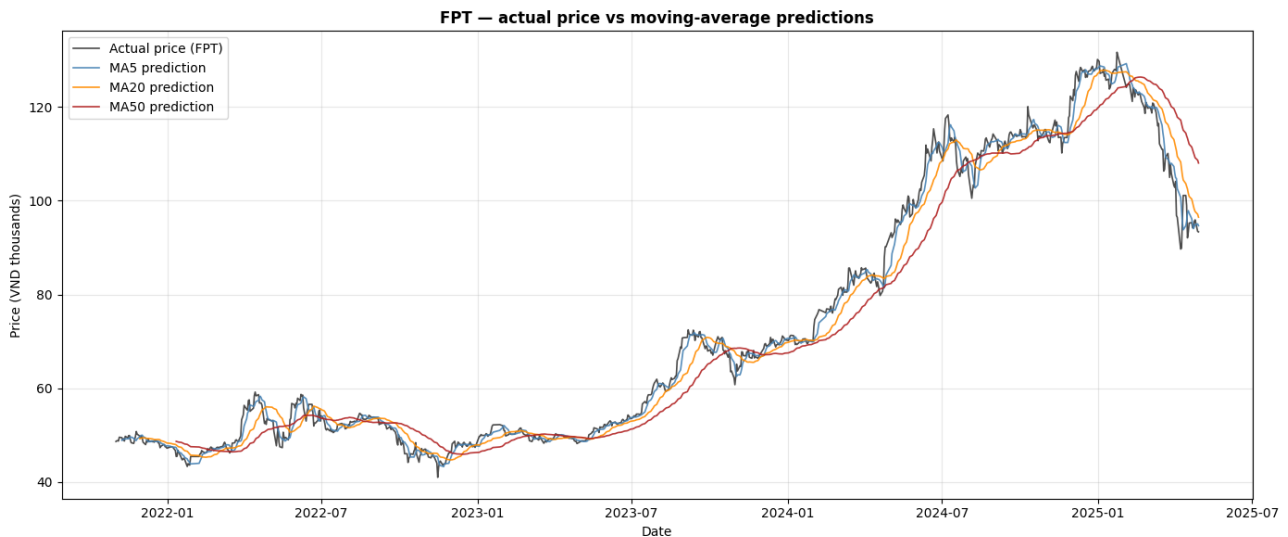


Figure 6: FPT 실제 가격 대비 MA5, MA20, MA50 예측값

MA5 는 실제 가격을 비교적 밀접하게 추적하는 반면, MA50 은 더 부드럽지만 가격 전환점에서 반응이 매우 느리다.

적절한 평균 일수를 선택하기 위해, 데이터를 시간 순서대로 앞 80% 는 훈련 세트, 뒤 20% 는 테스트 세트로 분리한 뒤, 테스트 세트에서의 예측 오차를 MAPE(평균 절대 백분율 오차, 실제 가격 대비 평균 오차 비율)로 측정하였다.

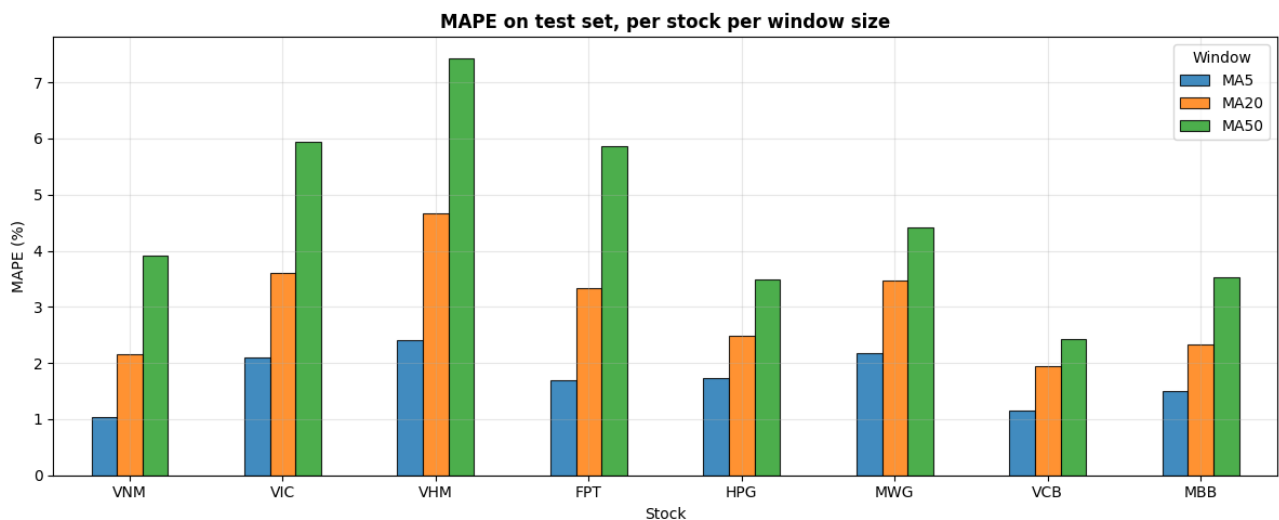


Figure 7: 종목별 MA5, MA20, MA50 의 MAPE 비교

MA5 는 모든 종목에서 가장 낮은 오차를 기록하였으며, 평균 MAPE 는 약 1.7% 로 MA20(3.0%) 과 MA50(4.6%) 보다 우수하였다. 짧은 기간의 평균일수로서 가격 변동에 빠르게 반응하기 때문이다. 따라서 본 프로젝트는 MA5 를 이후 최적화 단계의 예측 기준선으로 채택하였다. 종목별로는 VNM 이 변동성이 낮아 가장 예측하기 쉬웠으며 (오차 약 1%), VHM 은 부동산 업종의 어려움으로 인한 높은 변동성 때문에 가장 예측하기 어려웠다.

다만 MA5 는 내일 가격을 비교적 정확하게 예측하지만 (모든 종목에서 오차 2.5% 미만), 이 방법이 추정하는 기대 수익률은 본질적으로 최근 5 일간의 단기 추세를 그대로 연장한 것에 불과하

다. 따라서 MA 기반 기대 수익률은 비교 기준선으로만 활용하며, 보다 신뢰도 높은 추정을 위해 다음 절에서 다중선형회귀를 적용한다.

### 3.5 다중선형회귀 예측 — MLR (NB05)

이동평균법의 가장 큰 한계는 각 종목의 과거 데이터만을 독립적으로 활용하여, PCA 단계에서 공들여 발견한 종목 간 상관관계를 전혀 활용하지 못한다는 점이다. 다중선형회귀 (Multiple Linear Regression, MLR) 는 PCA 에서 도출한 세 가지 잠재 요인을 직접 예측 변수로 활용함으로써 이 한계를 극복한다.

구체적으로, 각 종목에 대해 내일 가격을 네 가지 변수, 즉 시장의 세 가지 잠재 요인 (PC1, PC2, PC3) 과 해당 종목의 당일 가격으로 예측한다.

$$P_{t+1}^{(i)} = \beta_0 + \beta_1 \text{PC1}_t + \beta_2 \text{PC2}_t + \beta_3 \text{PC3}_t + \beta_4 P_t^{(i)} \quad (5)$$

당일 가격  $P_t^{(i)}$  를 포함하는 이유는 주가의 연속성 때문이다. 내일 가격은 오늘 가격과 매우 가까운 경향이 있으므로, 이를 변수로 포함하면 예측 정확도가 크게 향상된다.

#### 3.5.1 수학적 기반

MLR 은 예측 오차의 제곱합  $\|y - X\beta\|^2$  을 최소화하는 계수 벡터  $\beta$  를 구하는 문제다. 이 최소화 문제는 선형대수학에서 잘 알려진 **정규방정식 (Normal Equation)** 으로 닫힌 형태의 해를 갖는다.

$$\beta = (X^T X)^{-1} X^T y \quad (6)$$

여기서  $X$  는 예측 변수들을 행으로 쌓은 설계 행렬 (design matrix) 이고,  $y$  는 예측 대상인 내일 가격 벡터다. 기하학적으로 보면, 예측값  $\hat{y} = X\beta$  는  $y$  를  $X$  의 열 공간에 직교 투영한 결과, 즉 선형 모델이 도달할 수 있는  $y$  에 가장 가까운 점이다. 본 프로젝트는 이 공식을 NumPy 로 직접 구현한 뒤 sklearn 라이브러리로 검증하였으며, 두 결과가 완전히 일치함을 확인하였다.

#### 3.5.2 핵심 코드

최종 목표는 다음 공식으로 내일 가격을 예측하는 것이다.

$$\hat{P}_{t+1} = \beta_0 + \beta_1 \text{PC1}_t + \beta_2 \text{PC2}_t + \beta_3 \text{PC3}_t + \beta_4 P_t \quad (7)$$

이 공식을 사용하려면 먼저 계수 벡터  $\beta$ , 즉 각 요인이 가격에 미치는 영향의 크기를 구해야 한다. 이것이 바로 정규방정식이 해결하는 문제다.

정규방정식은  $X^T X$  의 역행렬 계산을 필요로 하는데, 이를 직접 수행하면 수치 불안정성이 생길 수 있다. 따라서 역행렬을 명시적으로 계산하는 대신, 선형 연립방정식  $X^T X \beta = X^T y$  를 `np.linalg.solve` 로 직접 풀어  $\beta$  를 구한다. 이 방법은 역행렬 방법과 수학적으로 동일하지만 더 빠르고 정확하다.

Listing 5: 정규방정식을 이용한 MLR 구현 (NB05)

```
def fit_normal_equation(X, y):
    """Solve OLS via the Normal Equation:  $\beta = (X^T X)^{-1} X^T y$ .
    Uses np.linalg.solve for numerical stability:
    instead of computing inv(X^T X) explicitly,
    I solve the linear system  $(X^T X) \beta = X^T y$  directly."""
    X_np = X.values if hasattr(X, 'values') else X
    y_np = y.values if hasattr(y, 'values') else y

    XtX = X_np.T @ X_np # Gram matrix, shape (p, p)
    Xty = X_np.T @ y_np # shape (p,)
    beta = np.linalg.solve(XtX, Xty)
    return beta
```

각 종목에 대한 계수  $\beta$  를 구한 후, 당일의 예측 변수값을 공식에 대입하여 내일 가격을 계산한다. 이 과정을 전체 테스트 기간에 걸쳐 반복하면 예측 가격 시계열을 얻으며, 이를 실제 가격과 비교하여 모델 정확도를 평가한다.

### 3.5.3 Moving Average 와의 비교

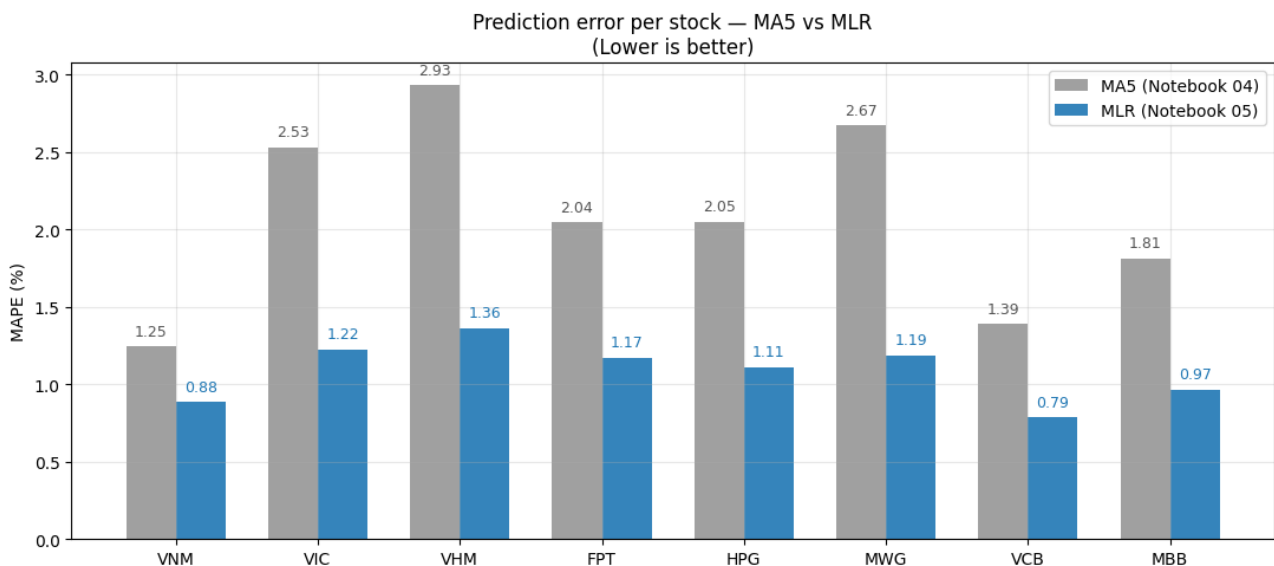


Figure 8: 종목별 MA5 와 MLR 의 MAPE 비교

MLR 은 8 개 종목 전체에서 MA5 를 능가하였다. 평균 MAPE 는 MA5 의 약 2.1% 에서 MLR 의 약 1.1% 로 절반 가까이 감소하였으며, 변동성이 큰 부동산 종목인 VHM 과 VIC 에서 개선 폭이 가장 두드러졌다.

다만 MLR 이 내일 가격 예측에서는 MA5 보다 정확하지만, 기대 수익률 추정에서는 여전히 한계가 있다. 모든 종목에서 회귀 절편이 약 0.99 에 달하는 것은 모델이 본질적으로 “내일 가격  $\approx$  오늘 가격” 을 학습했음을 의미하며, 이로부터 도출되는 기대 수익률은 테스트 기간의 실제 수익률을 그대로 반영할 뿐, 진정한 장기 예측이라 보기 어렵다. 이는 단기 가격 예측 기반 수익률 추

정의 구조적 한계이며, 두 방법 모두 내일 예측 오차는 작더라도 장기적으로 신뢰할 수 있는 기대 수익률을 제공하지 못한다는 점을 인식해야 한다.

### 3.6 Sharpe ratio 최적화 (NB06)

각 종목의 기대 수익률 (예측 단계에서 도출) 과 리스크를 나타내는 공분산 행렬 (분석 단계에서 도출) 을 확보한 후, 이제 8 개 종목에 자본을 어떻게 배분해야 Sharpe ratio 가 최대화되는지를 구한다.

#### 3.6.1 수학적 기반

각 배분 방식은 비중 벡터  $w = [w_1, \dots, w_8]$  으로 표현되며,  $w_i$  는 종목  $i$  에 투입하는 자본의 비율이다. 임의의 비중 벡터에 대해 포트폴리오의 기대 수익률은  $\mu^\top w$  로, 리스크는 이차형식  $w^\top \Sigma w$  로 계산된다. 최적화 문제는 다음과 같이 정식화된다.

$$\max_w \frac{\mu^\top w - r_f}{\sqrt{w^\top \Sigma w}} \quad \text{s.t.} \quad \sum_i w_i = 1, \quad w_i \geq 0 \quad (8)$$

비중의 합이 1 이라는 조건은 보유 자금의 100% 를 포트폴리오에 투입하되 현금을 보유하거나 추가 차입을 하지 않음을 의미한다. 비중이 음수가 될 수 없다는 조건은 공매도를 허용하지 않음을 의미한다. 무위험 이자율은 베트남 10 년 만기 국채 수익률에 해당하는  $r_f = 4\%$  로 설정하였다.

#### 3.6.2 핵심 코드

SciPy 는 최솟값을 구하도록 설계되어 있으므로, Sharpe ratio 의 최댓값을 구하기 위해 목적함수에 음의 부호를 붙여 최솟값 문제로 변환한다. SLSQP 알고리즘을 사용하여 제약 조건 하에서 최적 비중을 구한다.

Listing 6: SciPy SLSQP 를 이용한 포트폴리오 최적화 (NB06)

```
def optimize_portfolio(mu, sigma, rf=RISK_FREE_RATE, verbose=True):
    """Find weights that maximize Sharpe ratio (long-only, fully invested)"""
    result = minimize(
        fun=neg_sharpe,
        x0=w_init,
        args=(mu, sigma, rf),
        method='SLSQP',
        bounds=bounds,
        constraints=[budget_constraint],
        options={'ftol': 1e-10, 'maxiter': 200, 'disp': verbose}
    )
    w_opt = result.x
    return w_opt, result
```



### 3.6.3 결과

최적 비중 결과를 보면, 최적화 알고리즘이 자동으로 2~3 개 종목에만 집중 투자하는 경향을 보인다. MLR 기대 수익률을 사용한 경우, FPT 에 71.8%, VNM 에 17.6%, VIC 에 10.7% 를 배분하고 나머지 5 개 종목에는 0% 를 할당하였다.

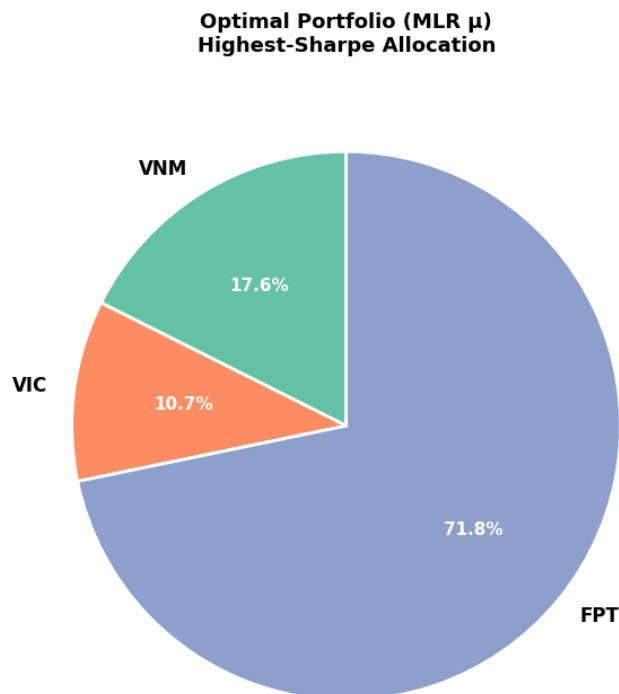


Figure 9: MLR 기대 수익률 기반 최적 포트폴리오 구성

이는 위험 대비 수익 기여도가 가장 높은 소수 종목에 자본을 집중시키고 나머지를 0 으로 만드는 최적화 알고리즘의 특성에서 비롯된다.



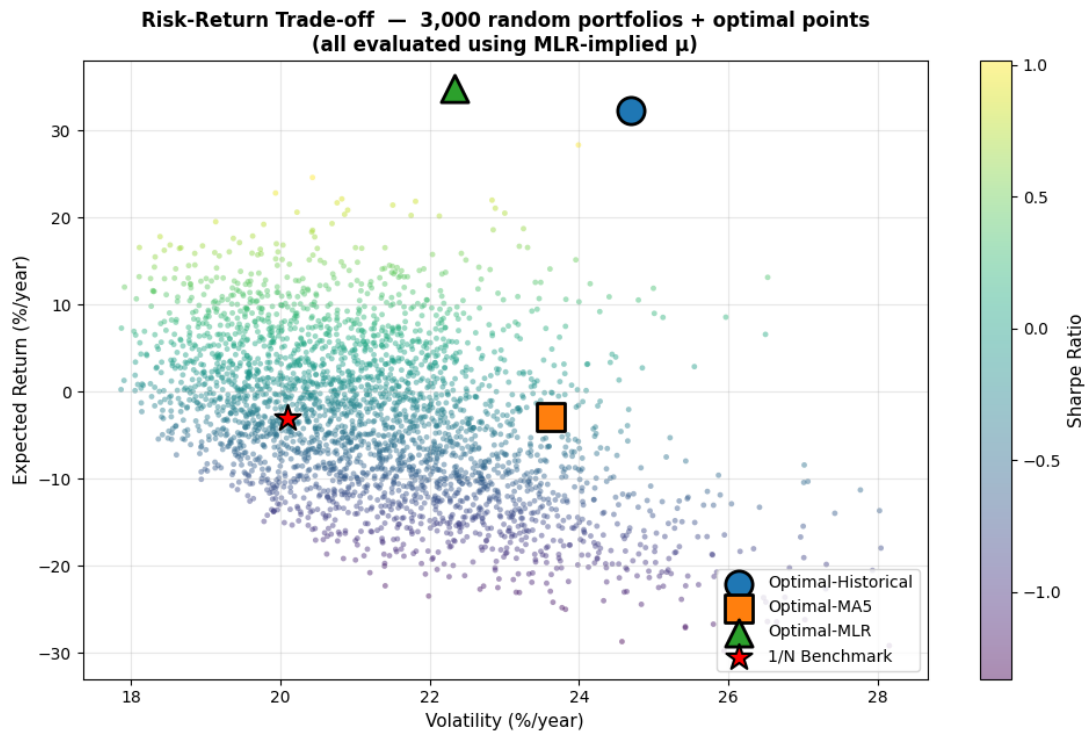


Figure 10: 리스크-수익률 관계도: 최적 포트폴리오 vs. 3,000 개 무작위 포트폴리오 및 균등 배분 벤치마크

리스크-수익률 산점도에서 각 점은 하나의 자본 배분 방식을 나타내며, 왼쪽 상단에 위치할수록 낮은 리스크로 높은 수익을 달성하는 우수한 포트폴리오임을 의미한다. 모든 최적화 포트폴리오가 균등 배분 벤치마크 (별 모양) 보다 높은 위치에 있어, 동일한 리스크 수준에서 더 높은 수익률을 달성하였다. 이는 이론적으로 기대되는 결과로, 신중한 비중 계산이 무작위 배분보다 우수한 성과를 낼 수 있음을 보여준다.

다만 이 결과는 표본 내 (in-sample) 평가임을 유의해야 한다. 즉, 기대 수익률과 리스크를 추정하는 데 사용한 것과 동일한 데이터로 평가한 것이다. 모델이 학습에 사용한 데이터로 직접 검증되므로 지나치게 낙관적인 평가가 될 수 있으며, 이에 대한 분석은 다음 장에서 다룬다.

## 4 문제점과 해결 과정

### 4.1 아웃오브샘플 백테스트

3 장에서 과거 데이터로 평가했을 때 최적화 포트폴리오는 높은 수익률과 낮은 리스크로 균등 배분 및 모든 무작위 포트폴리오를 압도하였다. 그러나 앞서 언급한 바와 같이, 이는 표본 내 (in-sample) 평가, 즉 모델이 학습한 데이터와 동일한 데이터로 검증한 결과이므로 지나치게 낙관적이다. 진정한 검증은 모델이 한 번도 본 적 없는 미래 데이터로 평가하는 아웃오브샘플 (out-of-sample) 백테스트여야 한다.

본 프로젝트는 데이터를 시간 순서대로 두 구간으로 분리하였다. 앞쪽 약 75%(2021 년 말 2024 년 중반) 는 기대 수익률과 리스크를 추정하고 최적 포트폴리오를 도출하는 데 사용하였다. 뒤쪽 약 25%(2024 년 중반 2025 년 초) 는 미래 구간으로 간주하여 모델 검증에만 활용하였다. 앞 구간에서 도출한 최적 포트폴리오를 뒤 구간에 그대로 적용하여 실제 손익을 측정하고, 균등 배분 벤

치마크와 비교하였다.

훈련 구간에서 최적화 포트폴리오의 Sharpe ratio 는 1.18 이었으며, FPT 에 92%, VCB 에 8% 를 집중 배분하였다.

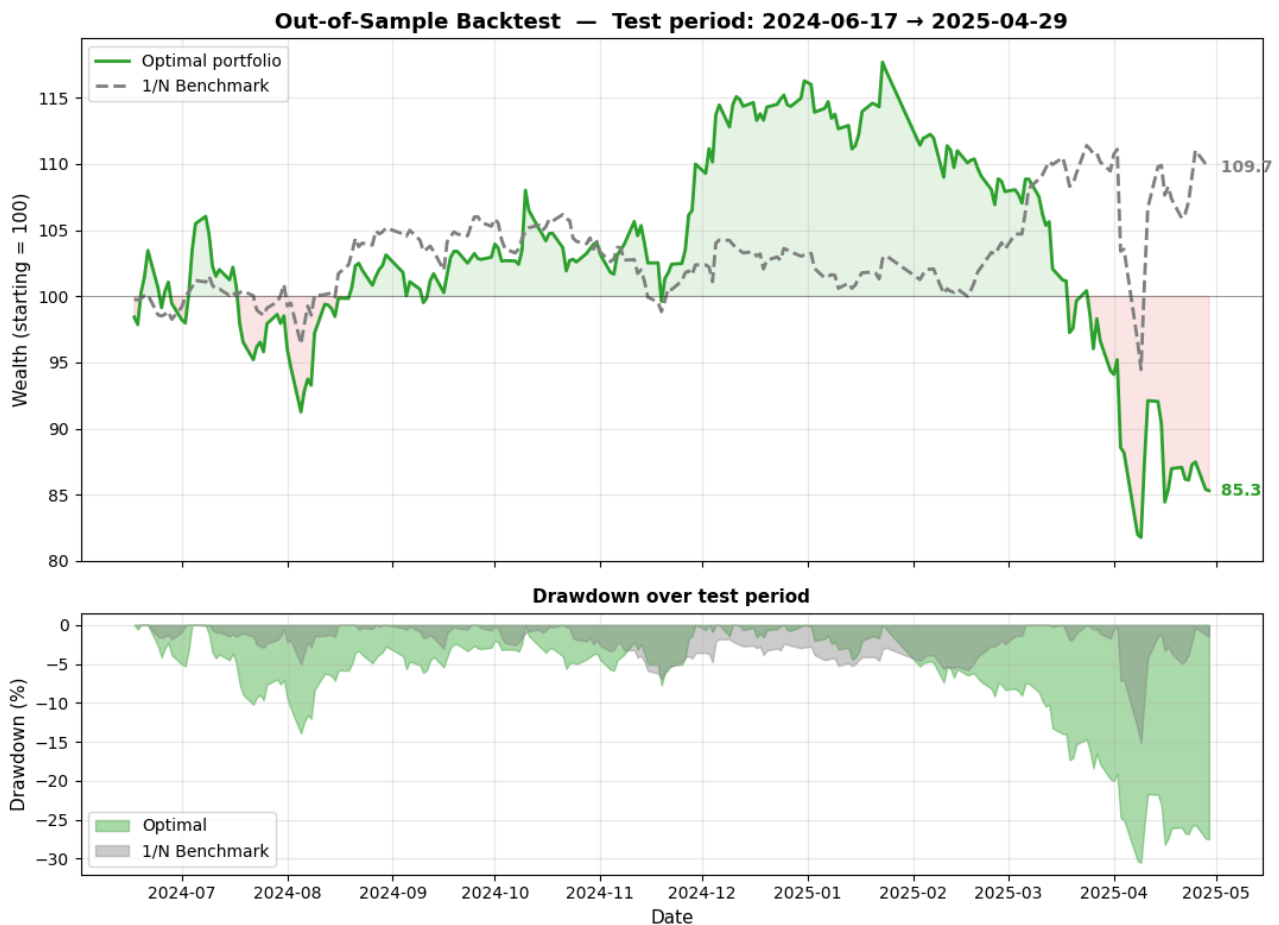


Figure 11: 테스트 구간에서의 최적 포트폴리오와 균등 배분 벤치마크의 자산 가치 추이 및 낙폭

100 단위 자금으로 시작했을 때, 테스트 구간 종료 시점에 최적화 포트폴리오는 85.3 으로 약 15% 손실을 기록한 반면, 균등 배분 포트폴리오는 109.7 로 약 10% 수익을 달성하였다.

## 4.2 과적합 및 추정 오차 분석

최적화 문제는 과거 데이터를 기반으로 결론을 도출한다. 최적화 포트폴리오가 FPT 에 92% 를 집중 배분한 이유는, 훈련 구간에서 FPT 가 높은 수익률과 최고의 Sharpe ratio 를 기록했기 때문이다. 그러나 알고리즘은 과거의 우수한 성과가 미래를 보장하지 않는다는 사실을 알지 못한다. 실제로 테스트 구간 진입 직후 FPT 는 2025 년 초 대규모 조정을 맞아 주가가 130,000 동대에서 95,000 동 아래로 급락하였다.

이는 과적합 (overfitting) 현상으로, 모델이 지속 가능한 패턴을 학습하는 대신 과거 데이터의 우연한 특성에 지나치게 맞춰진 결과다. 어떤 종목이 운이 좋아 과거 수익률이 높게 나왔더라도, 알고리즘은 즉시 해당 종목에 자본을 집중시킴으로써 작은 추정 오차를 극단적인 배분 결정으로 증폭시킨다.

반면 균등 배분은 어떤 종목이 더 나올지 예측하려 하지 않는다. 자본을 모든 종목에 고르게 분산하므로 특정 종목이 급락하더라도 큰 손실을 입지 않는다.

### 4.3 추정 오차의 영향

위의 결과는 방법론의 부적절함을 의미하는 것이 아니라, 포트폴리오 최적화 문제가 그 수식적 표현보다 훨씬 복잡하다는 사실을 보여준다.

핵심적인 어려움은 기대 수익률  $\mu$ 의 추정에 있다. 최적화 문제의 두 입력값은 기대 수익률  $\mu$ 와 공분산 행렬  $\Sigma$ 다. 이 중  $\Sigma$ 는 시간에 따라 비교적 안정적이며 허용 가능한 신뢰도로 추정할 수 있다. 반면  $\mu$ 는 정확한 추정이 매우 어렵다. 따라서  $\mu$  추정의 작은 오차가 포트폴리오 구성의 큰 변화로 이어질 수 있다.

## 5 한계 및 개선 방향

### 5.1 현재 모델의 한계

본 프로젝트가 달성한 성과와 더불어, 몇 가지 주요 한계점이 존재한다.

첫째, 기대 수익률  $\mu$  추정의 어려움이다. 사용된 두 예측 방법인 이동평균과 다중선형회귀 모두 과거의 단기 추세를 연장하는 방식이므로, 실제 수익률과 상당한 차이를 보이는  $\mu$  추정값을 산출한다. 최적화 결과가  $\mu$ 에 매우 민감하게 반응하는 만큼, 이 한계는 최종 포트폴리오의 품질에 직접적인 영향을 미친다.

둘째, 포트폴리오를 한 번만 최적화하고 고정한다. 본 프로젝트는 훈련 구간에서 파라미터를 추정하고 테스트 구간 전체에 걸쳐 동일한 포트폴리오 구성을 유지하며 정기적인 리밸런싱을 수행하지 않는다. 실제 시장에서는 조건이 지속적으로 변화하므로 정적 포트폴리오는 시간이 지남에 따라 효율성을 유지하기 어렵다. 이러한 접근 방식은 4장에서 분석한 것처럼 비중이 큰 자산이 불리하게 움직일 때 포트폴리오가 취약해지는 문제를 야기하기도 한다.

셋째, 데이터의 범위가 제한적이다. 본 프로젝트는 약 3년간 HOSE에 상장된 8개 블루칩 종목만을 분석 대상으로 삼았다. 적은 수의 자산은 분석의 명확성을 높이지만, 특히 3.2절에서 확인한 것처럼 모든 종목 간 상관관계가 양의 값을 가질 때 분산투자 여지를 제한한다. 또한 3년이라는 기간은 안정적인 통계 추정을 위해 비교적 짧은 편이다.

넷째, 모델이 일부 현실적 요소를 무시한다. 본 프로젝트는 거래 비용이 없다고 가정하며, 유동성이나 세금 등 다른 현실적 제약도 고려하지 않는다. 이러한 요소들이 포함될 경우 최적 배분 결과가 크게 달라질 수 있다.

### 5.2 개선 방향

위의 한계를 바탕으로, 향후 연구를 위한 몇 가지 발전 방향을 제시한다.

1. **기대 수익률  $\mu$  추정 개선:**  $\mu$ 가 핵심 취약점인 만큼, 이 부분의 개선이 가장 큰 잠재력을 지닌다. 선형 모델을 비선형 관계와 장기 시계열 의존성을 포착할 수 있는 LSTM 순환 신경망 등 더 표현력 높은 모델로 대체하는 것을 고려할 수 있다.

2. **최적화 문제의 안정화:** 최적화 알고리즘은 자본을 한 종목에 집중시키는 경향이 있다. 각 종목의 최대 비중을 30% 로 제한하는 등의 상한선을 설정하면 포트폴리오를 여러 종목에 분산시키도록 강제할 수 있어, 배분이 덜 극단적이고 추정 오차에 덜 민감해진다.
3. **정기적 포트폴리오 리밸런싱:** 고정된 포트폴리오를 유지하는 대신, 업데이트된 데이터를 바탕으로 매월 또는 매 분기 포트폴리오를 재최적화할 수 있다. 이 접근 방식은 변화하는 시장 환경에 포트폴리오가 적응할 수 있게 하며, 추가적인 거래 비용을 고려해야 한다는 점도 인식해야 한다.
4. **데이터 범위 확장:** 분석 대상 종목 수를 늘리고 데이터 기간을 연장하면 분산투자 여지가 확대되고 통계 추정의 신뢰도가 향상된다. 자산 수가 증가할수록 차원 축소에서 PCA 의 역할이 더욱 중요해진다.

### 5.3 결론

본 프로젝트는 베트남 주식 시장을 대상으로 데이터 수집 및 처리부터 리스크 분석과 PCA 를 통한 차원 축소, 이동평균과 다중선형회귀를 이용한 기대 수익률 예측, Sharpe ratio 최적화, 그리고 아웃오브샘플 포트폴리오 평가에 이르는 완전한 투자 포트폴리오 최적화 파이프라인을 구축하였다. 고유분해, 정규방정식, 제약 있는 최적화 등 핵심 수학적 도구를 모두 직접 구현하고 검증함으로써, 본 교과목의 수학적 기초에 대한 심층적 이해라는 목표를 달성하였다.

본 프로젝트의 가장 중요한 결과는 우수한 성과의 포트폴리오가 아니라, 방법론적 통찰에 있다. 아웃오브샘플 데이터에서 최적화 포트폴리오가 균등 배분보다 저조한 성과를 보인 사실은, 최적화 모델의 품질이 입력 데이터의 품질에 직접적으로 의존함을 보여준다. 가장 중요한 입력값인 기대 수익률을 예측하기 어려울 때, 모델의 복잡성이 반드시 실제 성과를 보장하지는 않는다.

## 참고문헌

- [1] Markowitz, H. M. (1991). *Foundations of Portfolio Theory*. *The Journal of Finance*, 46(2), 469–477. <https://www.jstor.org/stable/2328831>
- [2] Barreno, J., Skarpalezou, A., & Tisseverasinghe, A. (2020). *A Machine Learning Approach to Stock Price Prediction and Portfolio Optimization*. New York University Shanghai.